

A PROJECT REPORT

ON

LULC Change Detection Dashboard

**An AI-Powered Land Use / Land Cover Transition
Analysis System using OSCD-Enhanced ResUNet**

Submitted In Partial Fulfilment of the
Requirement for the Award of Post
Graduate Certificate in Artificial Intelligence
(PGCP-AI / PGDAI)

Under the Guidance of
(Project Guide Name)

Submitted By

Name	PRN
Deepak Srivastava	_____
Madhulika Srivastava	_____
Mitul Roy	_____
Arpita	_____

CDAC, B-30, Institutional Area,
Sector-62, Noida (Uttar Pradesh) – 201307

CERTIFICATE

CDAC, NOIDA

This is to certify that the Report entitled “LULC Change Detection Dashboard – An AI-Powered Land Use / Land Cover Transition Analysis System using OSCD-Enhanced ResUNet” which is submitted by Deepak Srivastava, Madhulika Srivastava, Mitul Roy and Arpita in partial fulfilment of the requirement for the award of Post Graduate Certificate in Artificial Intelligence (PGDAI / PGCP-AI) to CDAC, Noida is a record of the candidates’ own work carried out by them under my supervision.

The documentation embodies results of original work, and studies are carried out by the students themselves and the contents of the report do not form the basis for the award of any other degree to the candidates or to anybody else from this or any other institution.

(Project Guide)

ABSTRACT

Monitoring land use and land cover (LULC) change is essential for urban planning, environmental conservation, agriculture management, and sustainable development. Traditional change detection methods often provide only binary change / no-change maps and do not explain what each pixel changed from and to (for example, Forest → Urban). With advances in deep learning and freely available satellite imagery, intelligent systems can now estimate detailed land-cover transitions at scale.

This project, titled “LULC Change Detection Dashboard”, designs and implements an end-to-end AI system for LULC transition analysis on the Onera Satellite Change Detection (OSCD) dataset covering 24 cities worldwide. An OSCD-enhanced Residual U-Net (ResUNet) predicts six land-cover classes—Other, Water, Urban, Forest, Agriculture, and Barren—independently on two acquisition dates (t1 and t2). Transition statistics and maps are then derived by comparing the two predicted class maps. The model uses ten input channels (Sentinel-2 bands B04, B03, B02, B08, B11, B12 plus spectral indices NDVI, NDWI, NDBI, and BSI). Training labels are spectral pseudo-labels cleaned using official OSCD change masks.

The system includes a FastAPI backend that exposes REST endpoints for AOI listing and transition inference, and a React (Vite) dashboard that visualizes AOI locations, t1/t2 class percentages, transition summaries, and colorized prediction maps. Held-out evaluation of the OSCD-enhanced model achieved approximately 95.8% pixel accuracy and 0.755 mean Intersection-over-Union (mIoU) against cleaned enhanced labels (with test-time augmentation), substantially improving upon a 4-band RGB+NIR baseline (~71.4% accuracy / 0.518 mIoU). The dashboard was verified locally for all 24 OSCD AOIs. The project demonstrates a practical AI pipeline for satellite-based LULC transition monitoring and provides a foundation for future cloud deployment (e.g., Vercel frontend with a hosted API).

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and respect to our Project Guide, CDAC, Noida, for scholarly guidance, constructive suggestions, and expert supervision during the course of this project. Special thanks to the Program Coordinator for continuous support and academic direction throughout the PGDAI / PGCP-AI programme.

We are thankful to our project guide for constant discussions, encouragement of new ideas, and feedback that helped improve the model design, evaluation methodology, and dashboard usability. We also acknowledge the creators of the Onera Satellite Change Detection (OSCD) dataset for making multi-temporal Sentinel-2 imagery and change labels publicly available for research and education.

Finally, we thank our team members for collaborative effort in data preparation, model training, API development, frontend integration, testing, and documentation.

TABLE OF CONTENTS

CERTIFICATE	2
ABSTRACT	3
ACKNOWLEDGEMENT	4
TABLE OF CONTENTS	5
LIST OF FIGURES	6
LIST OF TABLES	7
CHAPTER 1: INTRODUCTION	8
1.1 Background	9
1.2 Objective	10
1.3 Motivation	11
1.4 Purpose and Scope	12
CHAPTER 2: SURVEY OF TECHNOLOGY	13
CHAPTER 3: FEASIBILITY STUDY	18
CHAPTER 4: REQUIREMENT AND ANALYSIS	20
CHAPTER 5: PRELIMINARY MODULE DESCRIPTION	23
CHAPTER 6: SYSTEM DESIGNING	26
CHAPTER 7: CODING	29
CHAPTER 8: RESULT	32
CHAPTER 9: CONCLUSION AND FUTURE SCOPE	35
REFERENCES	37

LIST OF FIGURES

FIG 1: Overall System Architecture of LULC Transition Dashboard

FIG 2: End-to-End Data and Inference Workflow (t1/t2 → Transition)

FIG 3: ResUNet Encoder–Decoder Architecture (10-channel input)

FIG 4: Dashboard – AOI Selector with 24 OSCD Cities

FIG 5: Dashboard – t1 vs t2 Land-Cover Percentage Cards (Mumbai)

FIG 6: Dashboard – Predicted t1/t2 LULC Maps and Transition Map

FIG 7: Land Cover Share Chart (Grouped t1 vs t2 Bars)

FIG 8: Transition Summary Panel (from → to percentages)

LIST OF TABLES

TABLE 1: LULC Class Taxonomy Used in the Project

TABLE 2: OSCD Official Train / Test City Split (24 AOIs)

TABLE 3: Input Feature Channels for OSCD-Enhanced ResUNet

TABLE 4: Software Requirements

TABLE 5: Hardware Requirements

TABLE 6: Held-out Test Evaluation Results

TABLE 7: Example Mumbai Transition Summary (Model Estimates)

TABLE 8: Major REST API Endpoints

CHAPTER – 1

1. INTRODUCTION

Land Use and Land Cover (LULC) information describes how terrestrial surfaces are occupied and utilized—water bodies, built-up areas, forests, croplands, barren land, and other categories. Accurate and timely LULC maps support urban planning, disaster management, climate studies, agricultural monitoring, and policy decisions related to sustainable development. Multi-temporal satellite imagery enables observation of the same geographic region at different dates, making it possible to quantify how landscapes evolve over time.

Classical remote-sensing change detection often focuses on binary change masks that indicate only whether a pixel changed, without identifying the semantic transition. For planners and environmental analysts, knowing that a region changed is useful, but knowing that forest converted to urban, or agriculture converted to barren land, is far more actionable. Deep learning semantic segmentation models, particularly U-Net and Residual U-Net (ResUNet) architectures, have shown strong performance for pixel-wise classification of satellite imagery and can be applied independently to each date to recover class-to-class transitions.

This project develops an AI-powered LULC Transition Dashboard using the Onera Satellite Change Detection (OSCD) dataset. The system predicts land-cover classes at two times (t_1 and t_2), computes from $\rightarrow$ to transition statistics, optionally intersects predictions with official OSCD change masks, and presents interactive visualizations through a modern web dashboard connected to a FastAPI inference service.

1.1 BACKGROUND

Remote sensing satellites such as Sentinel-2 provide freely available multispectral imagery at 10 m spatial resolution for key bands, enabling frequent Earth observation. The OSCD dataset supplies co-registered image pairs for 24 cities with official binary urban change labels. However, OSCD does not provide multi-class LULC ground truth. Therefore, this project constructs spectral pseudo-labels using vegetation, water, built-up, and soil indices, then cleans those labels with OSCD change masks to reduce noise in stable (no-change) regions.

Earlier baseline experiments using 4-channel RGB+NIR inputs achieved moderate accuracy. Incorporating SWIR bands and spectral indices, together with change-aware label cleaning, produced a substantially stronger ResUNet model. The application layer wraps this model in a reproducible inference pipeline and a user-facing dashboard so that non-expert users can explore AOI-level transitions without writing code.

1.2 OBJECTIVE

The primary objectives of the project are as follows:

1. To design and train an OSCD-enhanced ResUNet model for multi-class LULC segmentation using ten spectral feature channels.
2. To generate LULC maps independently for t1 and t2 and derive semantic from → to transition statistics and visualization maps.
3. To develop a FastAPI backend exposing REST endpoints for AOI discovery and transition inference.
4. To build a React-based interactive dashboard showing 24 OSCD AOIs, t1 vs t2 class percentages, transition summaries, and prediction previews.
5. To evaluate model performance using pixel accuracy and mean IoU on held-out test AOIs, and compare against a baseline 4-band model.
6. To enable local deployment for demonstration and document a path for subsequent cloud deployment (e.g., Vercel frontend).

1.3 MOTIVATION

Urban expansion, deforestation, and agricultural conversion are among the most visible drivers of environmental and socio-economic change. Manual mapping of these transitions across many cities is expensive and slow. Binary change products help localize change hotspots but do not answer the critical “from-to” question. There is therefore a strong need for accessible AI tools that convert multi-temporal satellite stacks into interpretable LULC transition analytics.

The motivation for this project also comes from the availability of open datasets (OSCD, Sentinel-2) and mature deep learning frameworks (PyTorch), which make it feasible to build a complete research-to-demo pipeline within an academic project timeline. By packaging inference into a dashboard, the work bridges the gap between model training notebooks and usable decision-support prototypes.

1.4 PURPOSE AND SCOPE

The purpose of the LULC Change Detection Dashboard is to provide a preliminary, AI-assisted analytical platform for exploring land-cover transitions over OSCD cities. Users can select an Area of Interest (AOI), run the model, and inspect class fractions at t1 and t2, major transitions, and colorized maps.

Scope of the project includes:

- 24 OSCD cities (14 train / 10 test according to official OSCD split files).
- Six LULC classes: Other, Water, Urban, Forest, Agriculture, Barren.
- Single-date ResUNet segmentation applied to both dates (not a Siamese change network).
- Optional filtering of transitions using official OSCD binary change masks.
- Local FastAPI + React deployment; cloud deployment documented for later stages.

Limitations: LULC labels are pseudo-labels (not surveyed ground truth); Barrens remain weak due to scarce stable samples; outputs are model estimates for research/demo use and should not be treated as official land-cover products. The system currently focuses on OSCD AOIs rather than arbitrary user-uploaded geographies (though an upload endpoint for enhanced tensors exists).

CHAPTER – 2

2. SURVEY OF TECHNOLOGY

This chapter surveys the technologies used to build the LULC Transition Dashboard, including remote sensing data, deep learning for semantic segmentation, spectral indices, backend/frontend frameworks, and deployment tooling.

2.1 ARTIFICIAL INTELLIGENCE AND DEEP LEARNING

Artificial Intelligence enables systems to learn patterns from data and automate perception tasks. In remote sensing, deep convolutional neural networks (CNNs) are widely used for pixel-wise classification. Encoder–decoder networks such as U-Net capture both local texture and broader spatial context through skip connections. Residual U-Net (ResUNet) improves gradient flow and representation capacity by incorporating residual blocks, which is beneficial for complex multispectral scenes.

2.2 MACHINE LEARNING / SUPERVISED SEGMENTATION

This project uses supervised semantic segmentation. Each pixel is assigned one of six class IDs. Training minimizes a classification loss (cross-entropy / related objectives) over labeled tiles or full AOI crops. Evaluation uses pixel accuracy and mean Intersection-over-Union (mIoU). Test-time augmentation (TTA)—averaging predictions over flipped inputs—further improves robustness at inference time.

2.3 REMOTE SENSING AND OSCD DATASET

The Onera Satellite Change Detection dataset provides Sentinel-2 image pairs (imgs_1_rect / imgs_2_rect) for 24 cities and official binary change masks. Imagery is rectified to 10 m resolution. Official train.txt and test.txt define the city split. Although OSCD targets urban change detection, its multi-temporal stacks are highly suitable as inputs for LULC transition analysis when paired with class pseudo-labels.

2.4 SPECTRAL INDICES

Spectral indices compress band reflectance into features correlated with biophysical properties. This project uses:

- $NDVI = (B08 - B04) / (B08 + B04)$ — vegetation vigor
- $NDWI = (B03 - B08) / (B03 + B08)$ — water presence
- $NDBI = (B11 - B08) / (B11 + B08)$ — built-up / impervious surfaces
- BSI — bare soil index using SWIR/Red/NIR/Blue combinations

These indices, together with raw bands, form the 10-channel enhanced feature tensor consumed by ResUNet.

2.5 ResUNet ARCHITECTURE (NOT SIAMESE)

A Siamese network typically processes two dates jointly to output a change probability. This project intentionally does not use a Siamese U-Net as the primary model. Instead, one ResUNet classifies t1, the same weights classify t2, and transitions are computed by comparing class maps. This design directly yields from → to semantics required by the dashboard. An experimental early-fusion binary change script exists in the repository but is not used in the product path.

2.6 PyTorch

PyTorch is used for model definition, training, checkpointing, and GPU/CPU inference. It provides flexible tensor operations, automatic differentiation, and easy integration with NumPy for preprocessing and postprocessing.

2.7 FastAPI BACKEND

FastAPI is a modern Python web framework for building high-performance REST APIs with automatic OpenAPI documentation. In this project, FastAPI serves /status, /aois, and /predict/transition/{aoi}, returns JSON statistics, and hosts generated PNG artifacts via a static /uploads mount. CORS middleware enables browser access from the Vite development and preview servers.

2.8 REACT + VITE FRONTEND

The dashboard is implemented with React and Vite for fast development and optimized production builds. Leaflet is used for AOI map markers. Chart.js visualizes t1 vs t2 land-cover shares. The UI communicates with the API through a configurable VITE_API_BASE URL, which later supports Vercel deployment against a remote backend.

2.9 DATA STORAGE AND ARTIFACTS

Enhanced feature arrays may be cached as .npy files. Model checkpoints (.pt), masks, transition JSON, and colorized PNG maps are stored under `landcover_train/` and `backend/uploads/`. Large binaries are managed with Git LFS where appropriate. GitHub is used for version control and collaboration.

2.10 DEPLOYMENT TECHNOLOGIES

Local deployment uses uvicorn for the API and npm/Vite for the UI. For public hosting, the recommended path is a Dockerized API (Hugging Face Spaces / Render) and a Vercel frontend with `VITE_API_BASE` pointing to the public API URL. Containerization ensures reproducible environments for PyTorch dependencies.

CHAPTER – 3

3. FEASIBILITY STUDY

A feasibility study assesses whether the proposed LULC Transition Dashboard can be successfully implemented within academic resource constraints. Technical, economic, and behavioural dimensions are examined below.

3.1 TECHNICAL FEASIBILITY

The project uses mature, well-documented technologies: Python, PyTorch, FastAPI, React/Vite, NumPy, Pillow, and tiffle. OSCD imagery and labels are publicly available. Training can run on GPU for speed, while inference for a single AOI is practical on CPU for demonstration (though slower). The modular repository structure separates dataset building, training, evaluation, API, and UI, which improves maintainability. Hence the system is technically feasible.

3.2 ECONOMIC FEASIBILITY

Development relies primarily on open-source libraries and free datasets, minimizing licensing cost. Local demonstration requires only a standard laptop and free software toolchains. Optional cloud hosting (Vercel free tier for frontend; free/low-cost API hosts) keeps deployment expenses low for academic demos. Therefore the project is economically feasible.

3.3 BEHAVIOURAL FEASIBILITY

The dashboard interaction model is simple: select AOI → run transition → inspect t1/t2 percentages, charts, and maps. This workflow is accessible to students, researchers, and planners without requiring deep learning expertise. Clear online/offline API status and error messages further improve usability. The system is therefore behaviourally feasible for its target academic and demonstration audience.

CHAPTER – 4

4. REQUIREMENT AND ANALYSIS

Requirement analysis identifies functional needs, constraints, and resources required to deliver a working LULC transition system.

4.1 PROBLEM DEFINITION

Binary satellite change products do not explain semantic land-cover transitions. Training multi-class LULC models is difficult because OSCD lacks official multi-class labels. Users also need an interactive interface rather than command-line scripts. The problem addressed by this project is to build an end-to-end pipeline that (a) learns LULC segmentation from cleaned pseudo-labels, (b) estimates from → to transitions for OSCD cities, and (c) presents results in a web dashboard with local deployability.

4.2 FUNCTIONAL REQUIREMENTS

7. List all 24 OSCD AOIs split into train/test groups.
8. Load or build 10-channel features for selected AOI t1/t2.
9. Run ResUNet inference with optional TTA and OSCD mask filtering.
10. Return transition matrix, top transitions, t1/t2 class fractions, and map artifacts.
11. Display dual t1/t2 percentage cards, charts, map markers, and preview images in the UI.
12. Indicate API online/offline status and provide actionable error messages.

4.3 NON-FUNCTIONAL REQUIREMENTS

- Usability: clear AOI selection and labeled t1/t2 statistics.
- Performance: acceptable single-AOI inference latency for demo use.
- Portability: runs on Windows/Linux with documented scripts.
- Maintainability: modular backend services and React components.
- Transparency: document pseudo-label limitations and evaluation metrics.

4.4 SOFTWARE REQUIREMENTS

TABLE 4: Software Requirements

Component	Requirement
Language	Python 3.10+
DL Framework	PyTorch ≥ 2.1
API Framework	FastAPI, Uvicorn
Image/Array libs	NumPy, Pillow, tifffile

Frontend	Node.js 18+, React, Vite, Leaflet, Chart.js
Version Control	Git, Git LFS, GitHub
IDE / Tools	VS Code / Cursor, modern web browser
OS	Windows / Linux / macOS

4.5 HARDWARE REQUIREMENTS

TABLE 5: Hardware Requirements

Resource	Minimum	Recommended
CPU	Intel i5 / Ryzen 5 equivalent	Intel i7 / Ryzen 7 or better
RAM	8 GB	16 GB+
Storage	20 GB free (code + sample data)	50 GB+ with full OSCD + caches
GPU	Not mandatory for demo inference	NVIDIA GPU for training
Network	Required for npm/GitHub/LFS	Stable broadband

CHAPTER – 5

5. PRELIMINARY MODULE DESCRIPTION

The system is organized into modules that transform raw OSCD imagery into interactive transition analytics.

5.1 DATA PREPARATION MODULE

Located primarily in `landcover_train/build_oscd_enhanced_dataset.py`. It reads OSCD `imgs*_rect` bands, computes spectral indices, builds 10-channel feature tensors, generates/cleans pseudo-label masks using OSCD change labels, and writes enhanced dataset metadata. This module can be re-run after cloning when large `.npy` caches are gitignored.

5.2 MODEL TRAINING AND EVALUATION MODULE

`train_oscd_enhanced.py` defines the ResUNet architecture and training loop. `evaluate_oscd_enhanced.py` measures pixel accuracy and mIoU on held-out AOIs, with optional TTA. Checkpoints are saved under `landcover_train/train/`, notably `resunet_oscd_enhanced_best.pt`.

5.3 LULC TRANSITION MODULE

`lulc_transition.py` loads t1/t2 features (from enhanced cache or on-the-fly from OSCD), runs predictions, computes transition statistics, colorizes LULC and transition maps, and writes JSON/PNG outputs. It supports optional OSCD change-mask intersection so reported transitions emphasize officially labeled change regions.

5.4 BACKEND API MODULE

`backend/main.py` and `services/lulc_transition_service.py` expose REST endpoints, manage CORS, cache the loaded model, and store inference artifacts under `backend/uploads/transitions/`.

5.5 DASHBOARD UI MODULE

React components under `lulc-dashboard/src/Components/` implement the interactive experience: Dashboard orchestration, UploadPanel (AOI run controls), StatsCards (t1/t2 percentages), StatsChart, MapView, ChangePanel, and Sidebar layer toggles.

5.6 CONFIGURATION AND CONSTANTS MODULE

lulcConstants.js stores class colors, AOI coordinates, and official OSCD train/test lists so the UI can show all 24 cities even if the API is temporarily offline.

CHAPTER – 6

6. SYSTEM DESIGNING

System design defines architecture, data flow, and interfaces among modules. The LULC Transition Dashboard follows a layered client–server architecture.

6.1 ARCHITECTURE OVERVIEW

13. Presentation Layer: React dashboard (AOI selection, charts, maps, previews).
14. Application/API Layer: FastAPI routes and transition service.
15. AI/Inference Layer: ResUNet checkpoint + lulc_transition pipeline.
16. Data Layer: OSCD images/labels, enhanced features, uploads/artifacts.

FIG 1 (conceptual): Browser → FastAPI → ResUNet Inference → PNG/JSON Artifacts → Dashboard visualization.

6.2 SYSTEM WORKFLOW

The operational flowchart is:

17. Start: Launch API (uvicorn) and UI (Vite).
18. UI requests GET /status and GET /aois.
19. User selects AOI and options (OSCD mask filter, TTA).
20. UI calls GET /predict/transition/{aoi}.
21. Backend loads t1/t2 features (cache or OSCD on-the-fly).
22. ResUNet predicts class maps for t1 and t2.
23. Optional OSCD change mask is applied for transition selection.
24. Transition matrix, fractions, and color maps are generated and saved.
25. JSON response + artifact URLs are returned to UI.
26. Dashboard renders t1/t2 cards, chart, transition panel, and image previews.

6.3 CLASS TAXONOMY

TABLE 1: LULC Class Taxonomy Used in the Project

ID	Class	Typical Color
0	Other	Dark gray
1	Water	Dodger blue
2	Urban	Crimson
3	Forest	Forest green
4	Agriculture	Green-yellow
5	Barren	Tan

6.4 AOI SPLIT

TABLE 2: OSCD Official Train / Test City Split (24 AOIs)

Split	Cities
Train (14)	aguasclaras, bercy, bordeaux, nantes, paris, rennes, saclay_e, abudhabi, cupertino, pisa, beihai, hongkong, beirut, mumbai
Test (10)	brasilia, montpellier, norcia, rio, saclay_w, valencia, dubai, lasvegas, milano, chongqing

6.5 FEATURE DESIGN

TABLE 3: Input Feature Channels for OSCD-Enhanced ResUNet

#	Channel	Role
1–3	B04, B03, B02	Red, Green, Blue
4	B08	NIR
5–6	B11, B12	SWIR
7	NDVI	Vegetation
8	NDWI	Water
9	NDBI	Built-up
10	BSI	Bare soil

6.6 API DESIGN

TABLE 8: Major REST API Endpoints

Method	Endpoint	Purpose
GET	/status	Health and metadata
GET	/aois	Train/test AOI lists
GET	/predict/transition/{aoi}	Run LULC transition
POST	/predict/transition	Upload t1/t2 .npy tensors
GET	/uploads/...	Serve generated PNG/JSON artifacts

CHAPTER – 7

7. CODING

This chapter summarizes key implementation units. Full source code is maintained in the GitHub repository LULC-Change-Detection-Dashboard on branch lulc-transition-dashboard.

7.1 PROJECT STRUCTURE (SELECTED)

```
lulc-dashboard/                                #   React    +   Vite    UI
backend/                                       #   FastAPI   app
                                              main.py
services/lulc_transition_service.py

landcover_train/

                                              build_oscd_enhanced_dataset.py
                                              train_oscd_enhanced.py
                                              evaluate_oscd_enhanced.py
                                              lulc_transition.py
train/resunet_oscd_enhanced_best.pt

oscd_images/                                ...
oscd_labels/                                ...
scripts/start-backend.*                      /                      start-frontend.*
README.md, DEPLOY.md
```

7.2 FEATURE BUILDING (CONCEPTUAL)

Bands are loaded from OSCD imgs_*_rect GeoTIFF/TIFF files, scaled to reflectance, and stacked with NDVI/NDWI/NDBI/BSI into an (H, W, 10) float32 array. Missing enhanced cache files trigger on-the-fly feature construction during inference.

7.3 ResUNet INFERENCE (CONCEPTUAL)

The checkpoint loads ResUNet(in_channels=10). Input tensors are converted from HWC to NCHW, optionally padded to multiples of 16, passed through the network, and argmaxed to class IDs. With TTA, softmax probabilities from original and flipped views are averaged.

7.4 TRANSITION STATISTICS (CONCEPTUAL)

Given `pred_t1` and `pred_t2`, the system counts pixels for each (`from_class`, `to_class`) pair where classes differ (and optionally where OSCD change mask = 1). Percentages are reported relative to the full AOI and relative to changed pixels. Summary keys include `forest_to_urban`, `agriculture_to_urban`, `vegetation_loss`, etc.

7.5 FRONTEND DATA FLOW (CONCEPTUAL)

`Dashboard.jsx` fetches `/status` and `/aois` on mount. `UploadPanel.jsx` calls `/predict/transition/{aoi}`. `StatsCards.jsx` renders dual `t1/t2` values and percentage-point deltas. `StatsChart.jsx` draws grouped horizontal bars. `MapView.jsx` plots all AOI centers with Leaflet `CircleMarkers`.

7.6 LOCAL RUN COMMANDS

Backend:

```
cd backend && python -m pip install -r requirements.txt && python -m uvicorn main:app --host 127.0.0.1 --port 8000
```

Frontend:

```
cd lulc-dashboard && npm install && set VITE_API_BASE=http://127.0.0.1:8000 && npm run dev
```

CHAPTER – 8

8. RESULT

This chapter presents quantitative model results and qualitative dashboard outcomes observed during local deployment testing.

8.1 MODEL EVALUATION RESULTS

TABLE 6: Held-out Test Evaluation Results (with TTA where applicable)

Evaluation Target	Pixel Acc	mIoU	Urban IoU
OSCD-enhanced cleaned labels (best)	0.958	0.755	0.949
Same model vs legacy pseudo-labels	0.885	0.675	0.793
Previous 4-band RGB+NIR baseline	0.714	0.518	0.223

The OSCD-enhanced model clearly outperforms the earlier 4-band baseline, especially for Urban IoU. Barren remains the weakest class due to scarce stable pseudo-label pixels after change-aware cleaning.

8.2 DASHBOARD FUNCTIONAL RESULTS

- API /aois returns 14 train + 10 test = 24 cities.
- UI AOI dropdown lists all 24 cities in grouped optgroups.
- Stats cards display both t1 and t2 percentages with pp deltas.
- Transition run for Mumbai completed successfully in local testing.

8.3 EXAMPLE: MUMBAI TRANSITION (MODEL ESTIMATES)

TABLE 7: Example Mumbai t1/t2 Class Fractions (Model Estimates)

Class	t1 %	t2 %	Δ (t2-t1) pp
Other	16.4	12.9	-3.5
Water	1.7	1.3	-0.4
Urban	18.4	25.5	+7.1
Forest	23.2	17.2	-6.0
Agriculture	40.3	43.1	+2.8
Barren	0.0	0.0	0.0

Observed patterns for Mumbai in this demo run include increased Urban share and decreased Forest share between t1 and t2, consistent with urbanization signals in the predicted maps. When OSCD-mask filtering is enabled, reported change percentage focuses on officially labeled change regions (example run: ~1.52% of AOI pixels changed under mask).

8.4 ARTIFACT OUTPUTS

For each AOI run, the backend writes colorized t1 LULC, t2 LULC, and transition PNG maps plus a JSON summary under `backend/uploads/transitions/{aoi}/`. These artifacts are previewed in the dashboard after inference.

8.5 FIGURE REFERENCES (TO ATTACH IN FINAL PRINT)

- FIG 4: AOI dropdown showing 24 OSCD cities
- FIG 5: Mumbai t1 vs t2 percentage cards
- FIG 6: Predicted LULC and transition map previews
- FIG 7–8: Share chart and transition panel

Screenshots captured during local verification are available in the project artifacts and should be inserted into the printed report under Chapter 8.

CHAPTER – 9

9. CONCLUSION AND FUTURE SCOPE

9.1 CONCLUSION

The LULC Change Detection Dashboard successfully demonstrates an end-to-end AI pipeline for semantic land-cover transition analysis on the OSCD dataset. By combining an OSCD-enhanced 10-channel ResUNet with a FastAPI inference service and a React dashboard, the project converts multi-temporal satellite imagery into interpretable from → to analytics across 24 cities. Quantitative evaluation shows strong gains over a 4-band baseline, while the UI clearly presents t1 and t2 class percentages, maps, and transition summaries. The system is suitable for academic demonstration and as a foundation for further geospatial AI research.

9.2 FUTURE SCOPE

- Replace/augment pseudo-labels with ESA WorldCover or manually QA'd labels for stronger real-world accuracy.
- Improve Barren class performance or merge it into Other for operational products.
- Deploy public frontend on Vercel and host API on Hugging Face Spaces / Render / GPU cloud.
- Add support for user-uploaded AOIs with automatic Sentinel-2 retrieval and tiling.
- Optional hybrid use of binary change detectors only as attention/filters (not as replacements for class transition modeling).
- Add exportable PDF/CSV reports per AOI for planning stakeholders.
- Extend temporal analysis beyond two dates to multi-date trajectories.

REFERENCES

- [1] R. C. Daudt, B. Le Saux, and A. Boulch, “Fully Convolutional Siamese Networks for Change Detection,” in IEEE ICIP, 2018. (OSCD dataset)
- [2] Onera / OSCD project page: <https://rcdaudt.github.io/oscd/>
- [3] ESA, “Sentinel-2 User Handbook,” European Space Agency.
- [4] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” MICCAI, 2015.
- [5] Z. Zhang, Q. Liu, and Y. Wang, “Road Extraction by Deep Residual U-Net,” IEEE GRSL, 2018. (ResUNet concept)
- [6] PyTorch Team, “PyTorch Documentation,” <https://pytorch.org/docs/>
- [7] FastAPI, “FastAPI Documentation,” <https://fastapi.tiangolo.com>
- [8] Meta / React, “React Documentation,” <https://react.dev>
- [9] Vite, “Vite Documentation,” <https://vitejs.dev>
- [10] Leaflet, “Leaflet Documentation,” <https://leafletjs.com>
- [11] J. W. Rouse et al., “Monitoring Vegetation Systems in the Great Plains with ERTS,” NASA SP-351, 1974. (NDVI)
- [12] S. K. McFeeters, “The use of the Normalized Difference Water Index (NDWI) in the delineation of open water features,” IJRS, 1996.
- [13] Y. Zha, J. Gao, and S. Ni, “Use of normalized difference built-up index in automatically mapping urban areas from TM imagery,” IJRS, 2003. (NDBI)
- [14] ESA WorldCover, <https://esa-worldcover.org/>
- [15] Hugging Face, “Spaces Documentation,” <https://huggingface.co/docs/hub/spaces>
- [16] Vercel, “Vercel Documentation,” <https://vercel.com/docs>
- [17] Python Software Foundation, “Python 3 Documentation,” <https://docs.python.org/3/>
- [18] GitHub, “Git Large File Storage,” <https://git-lfs.com>